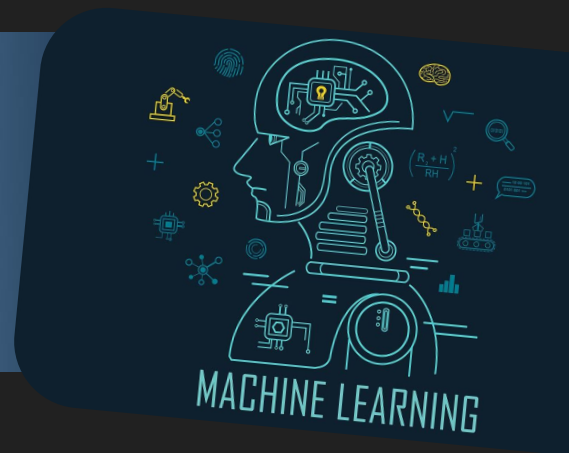




Foundations of Machine Learning
Joel Mattsson
1001343

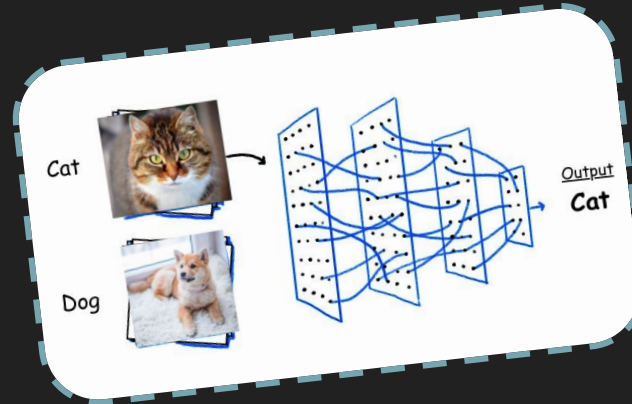
Problem Description:

Given a Kaggle dataset (<https://www.kaggle.com/datasets/rizkyyk/dataset-food-classification>), use a neural network for image classification

Dataset:

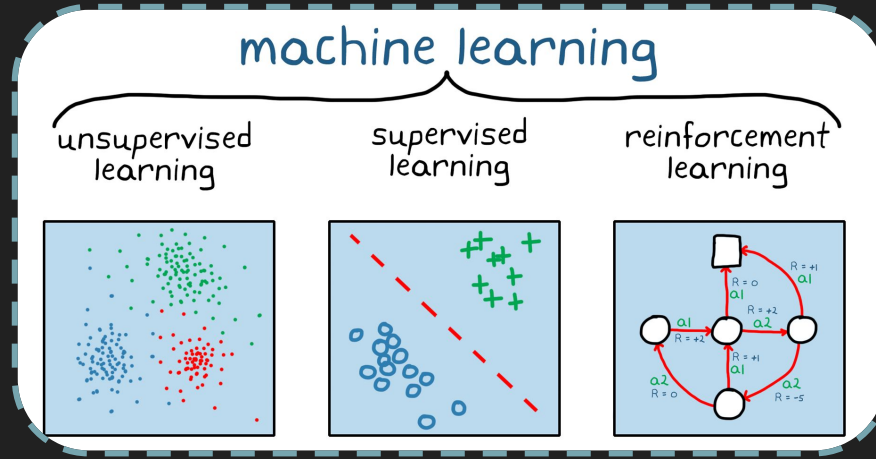
- 6500 images in total of popular Indonesian foods
- 13 unique types of foods (classes), each one containing 500 images
- **Classes:**
 - Ayam Goreng (Fried Chicken)
 - Burger
 - French Fries
 - Gado-Gado (Indonesian Salad)
 - Ikan Goreng (Fried Fish)
 - Mie Goreng (Fried Noodles)
 - Nasi Goreng (Fried Rice)
 - Nasi Padang (Padang Rice)
 - Pizza
 - Rawon (Beef Soup)
 - Rendang (Spiced Meat)
 - Sate (Satay)
 - Soto Ayam (Chicken Soup)
- **Directories:**
 - Training
 - Testing
 - Validation

kaggle



Problem Analysis:

- Splitting the dataset into training and testing sets is common for **Supervised Learning**
- **Classification** is based on a comparison of output and an associated ground truth

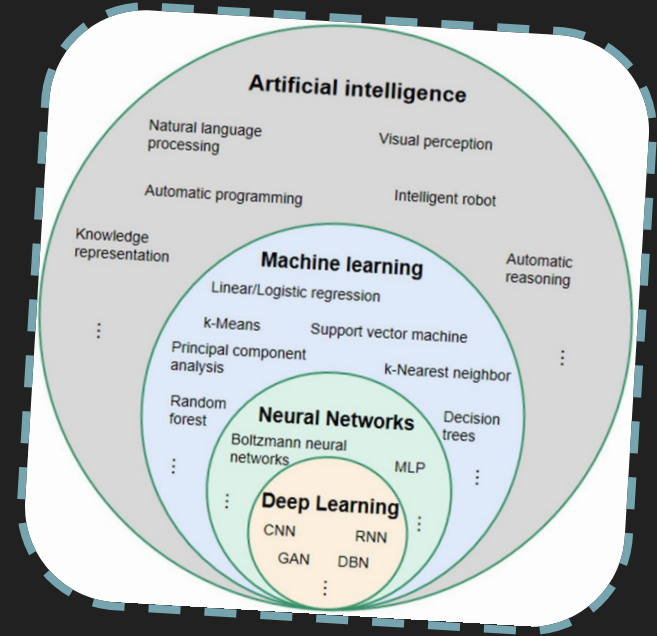


Conclusion:

- Implement a neural network to classify images that leverages **Supervised Learning**

Selection of Neural network type:

- **Models:**
 - *Supervised Learning: SVMs, k-Nearest, neighbor, ...*
 - *Unsupervised Learning: K-Means, PCA*
- **Neural Networks:**
 - *Recurrent Neural Networks: Handle sequential data by maintaining previous input*
 - *Multilayer Perceptrons: Can be used for image recognition but is limited in complexity*
 - *Convolutional Neural Networks: Image recognition*



Conclusion:

- **Convolutional Neural Networks** is the most suitable approach for the given problem, where complexity and accuracy is the priority

Neural Network Components

Most prominent

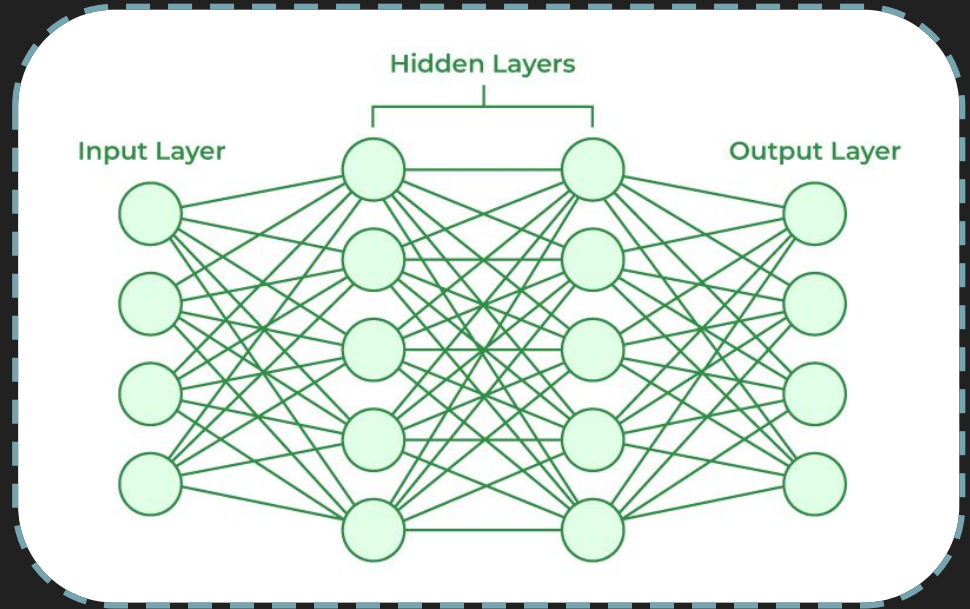
- *Neurons*
- *Weights*
- *Bias*

Backend

- *Activation Function*
- *Loss Function*
- *Optimizer*

Flow in Architecture

- *Forward Propagation*
- *Backward Propagation*



Convolutional Neural Networks

Most prominent

- Neurons
- Weights
- Bias

Backend

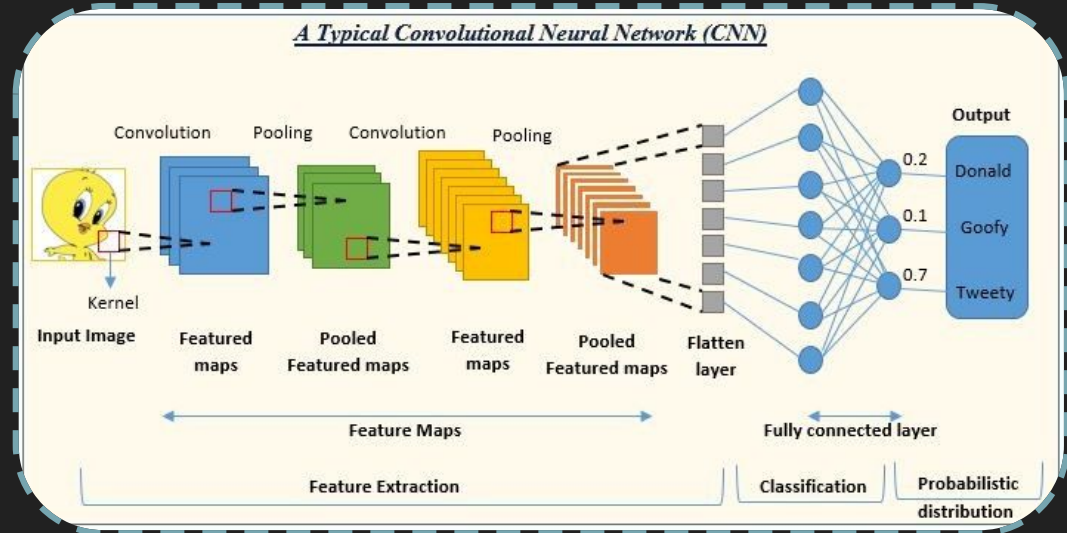
- Activation Function
- Loss Function
- Optimizer

Flow in Architecture

- Forward Propagation
- Backward Propagation

Extra Layers for CNN

- Convolutional Layers: Extract features such as edges and textures
- Pooling Layers: Reduce dimensions while retaining information to improve efficiency



Considered Models:

- ### Pros & Cons:

- ## Conclusion:

-

Coding Approach - Step 1: Data Exploration

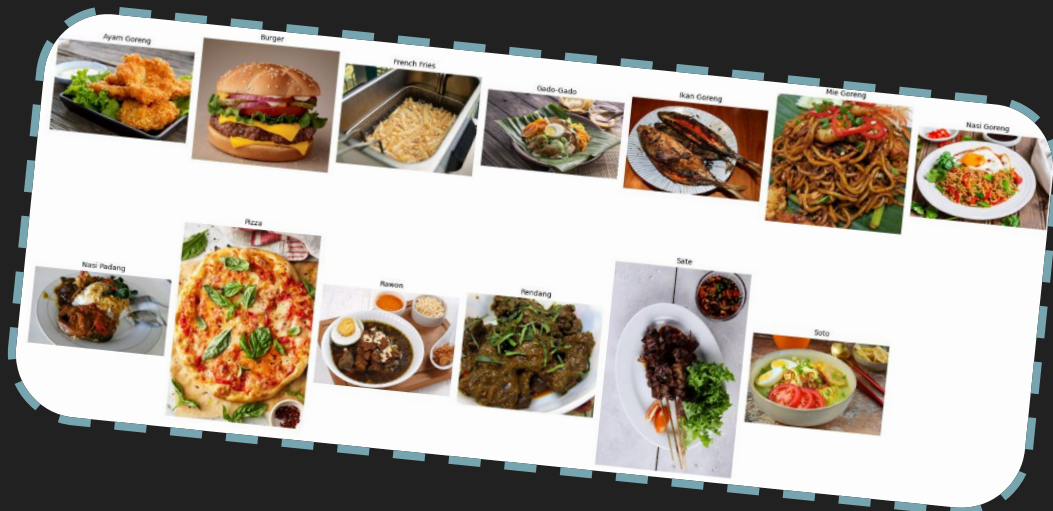
Information Theory

- **Entropy:** Measures the uncertainty of information of a variable. **Joint Entropy** and **Conditional Entropy** are concerned with **Feature Representation** and **Feature Selection**, as feature relationships constitute uncertainty in their relevance to the model's performance

Are there any feature relationships between the classes that can be used to distinguish between classes or cause overlapping?

- **Classes:**

- Ayam Goreng (Fried Chicken)
- Burger
- French Fries
- Gado-Gado (Indonesian Salad)
- Ikan Goreng (Fried Fish)
- Mie Goreng (Fried Noodles)
- Nasi Goreng (Fried Rice)
- Nasi Padang (Padang Rice)
- Pizza
- Rawon (Beef Soup)
- Rendang (Spiced Meat)
- Sate (Satay)
- Soto Ayam (Chicken Soup)



Coding Approach - Step 1: Data Exploration

Inspect the distribution of data in the used dataset for each of the classes and for the train, test and valid splits

For each class:

500 total

400 train

50 test

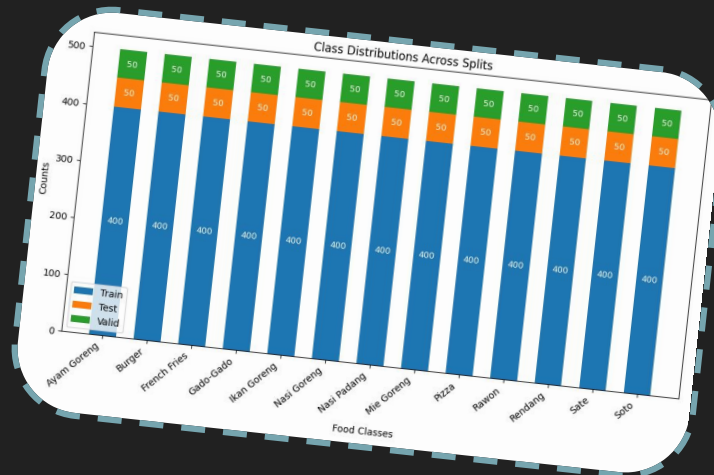
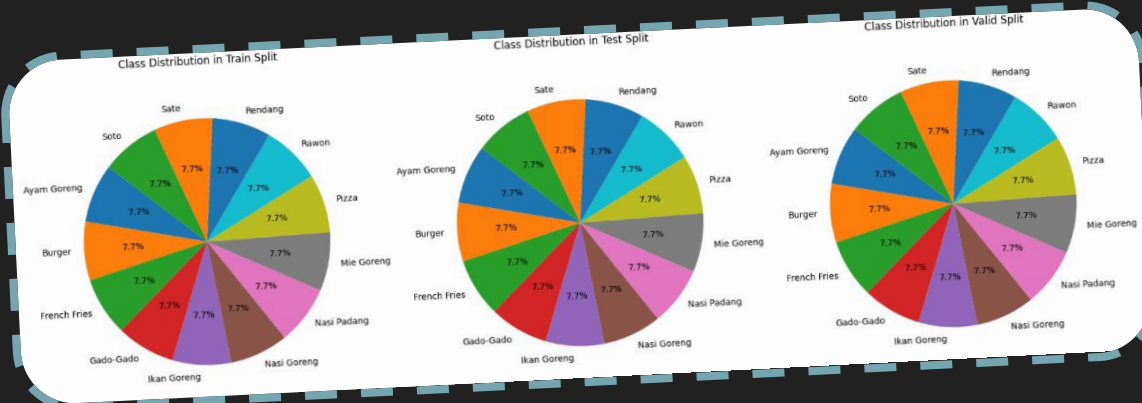
50 valid

Uniform distribution for all 13 classes:

$$100 / 13 = 7.7$$

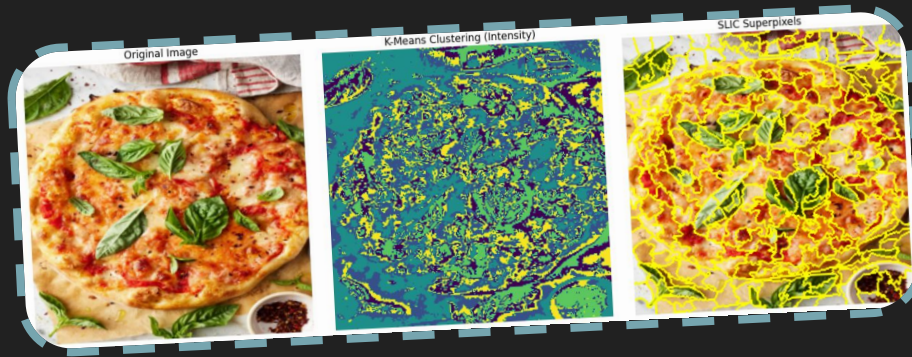
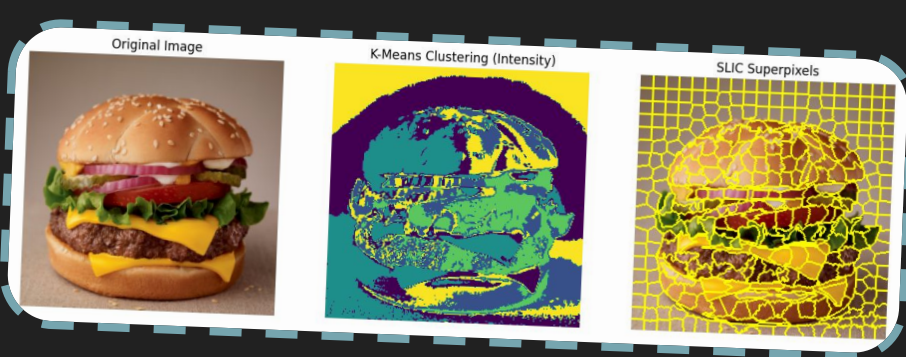
$$\text{train split} = 400 / 500 = 80\%$$

$$\text{test valid splits} = 50 / 500 = 10\%$$



Coding Approach - Step 1: Data Exploration

*Unsupervised Learning with Clustering to identify similarities and dissimilarities between the classes, using **K-Means** and **Normalized Cut***



Conclusions:

- *Understanding of how classes differ in pixel intensity, texture and region distribution*
- *Identify noisy data and inconsistencies*
- *Inform choice of model architecture*

Coding Approach - Step 2: Data Preprocessing

Code Implementations

- **Data Augmentation: Improves model generalization by simulating variations in the training data**

- Rotate images within a range of 30 degrees
- Shift image vertically and horizontally by 20%
- Flip images horizontally

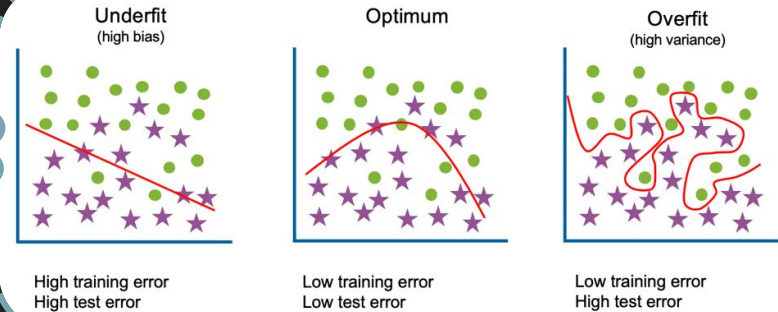
- **Early Stopping: Prevents overfitting and saves computational resources**

- In this project, the metric of loss from the loss function is monitored. If it doesn't improve for a certain number of epochs, the training stops and restores the model with peak performance

- **Learning Rate Scheduling: Dynamically adjusts learning rates to improve convergence**

- Use Adam Optimizer to control the size of the steps taken during optimization in the parameter space

Generalization Problem

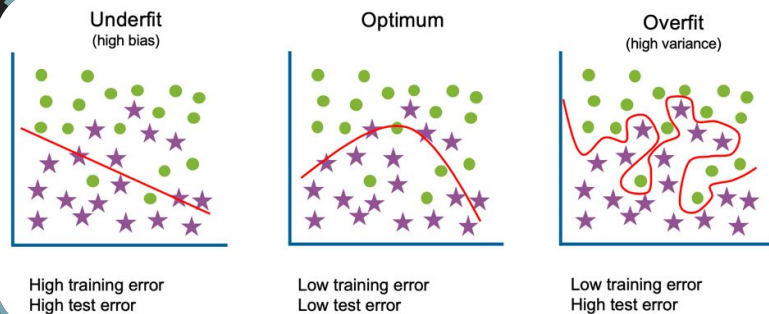


Coding Approach - Step 2: Data Preprocessing

L2 Regularization: Adds penalty terms to the loss function to discourage the model from learning overly complex patterns that leads to overfitting

Dropout: Randomly "drops out" (sets to zero) a subset of neurons during each training iteration, reducing the model's reliance on specific features and improving generalization

Generalization Problem



Coding Approach - Step 3: Model Training

Configured Settings for Coding Approach

1. Weights: Pre-trained from ImageNet
2. Activation Function = Softmax
3. Loss Function = Cross Entropy
4. Optimizer = Adam

Coding Approach - Step 3: Model Training

- **Softmax:** *Outputs a probability distribution for multi-classification problems. In the context of this project, since 13 images are classified into categories, the output layer has 13 neurons, each representing the probability that the input image belongs to a specific category*
- **Cross-Entropy:** *A loss function that compares predicted probability distributions with the true distributions. Note that these distributions are generated by **Softmax***
 - *A higher loss value prompts significant updates to model parameters and re-adjustments of weights*
 - *If the loss value increases, it indicates an inappropriate learning rate for the Optimizer component, or that the model is overfitting*

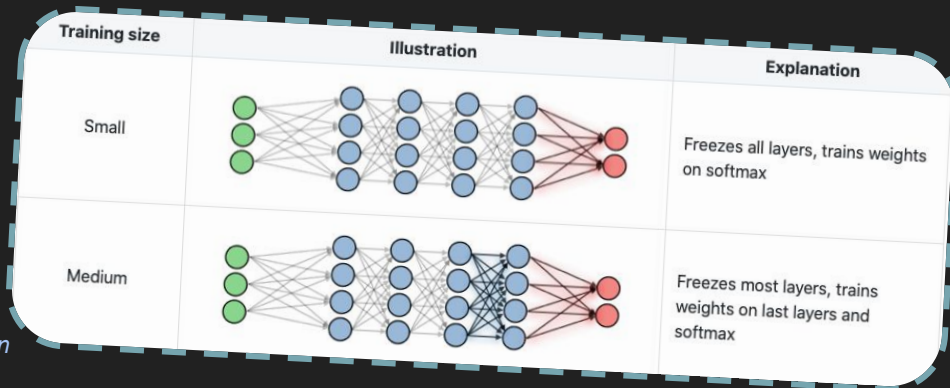
Coding Approach - Step 3: Model Training

Fine-tuning

- **Freezed Layers:** These layers won't be updated during training. Freezing is done to preserve the pre-trained weights
- **Transfer Learning:** Pre-trained weights allows you to adapt the model with smaller datasets by fine-tuning or freezing some layers
 - Saves time and resources

Trade-offs of Freezing

- **Pros:**
 - Faster training
 - Retention of general features learned from large datasets
- **Cons:**
 - Limits the model's capacity to fully adapt to new tasks
 - If the pre-trained weight doesn't align with the task, freezing can hinder performance



Layer 0: input_layer, Trainable: False
Layer 1: conv2d, Trainable: False
Layer 2: batch_normalization, Trainable: False
Layer 3: activation, Trainable: False
Layer 4: conv2d_1, Trainable: False
Layer 5: batch_normalization_1, Trainable: False
Layer 6: activation_1, Trainable: False
Layer 7: conv2d_2, Trainable: False
Layer 8: batch_normalization_2, Trainable: False
Layer 9: activation_2, Trainable: False
Layer 10: max_pooling2d, Trainable: False
Layer 11: conv2d_3, Trainable: False
Layer 12: batch_normalization_3, Trainable: False
Layer 13: activation_3, Trainable: False
Layer 14: conv2d_4, Trainable: False
Layer 15: batch_normalization_4, Trainable: False
Layer 16: activation_4, Trainable: False
Layer 17: max_pooling2d_1, Trainable: False
Layer 18: conv2d_8, Trainable: False
Layer 19: batch_normalization_8, Trainable: False
Layer 20: activation_8, Trainable: False
Layer 21: conv2d_6, Trainable: False
Layer 22: conv2d_9, Trainable: False
Layer 23: batch_normalization_6, Trainable: False
Layer 24: batch_normalization_9, Trainable: False
Layer 25: activation_6, Trainable: False
Layer 26: activation_9, Trainable: False
Layer 27: average_pooling2d, Trainable: False
Layer 28: conv2d_5, Trainable: False
Layer 29: conv2d_7, Trainable: False
Layer 30: conv2d_10, Trainable: False
Layer 31: conv2d_11, Trainable: False

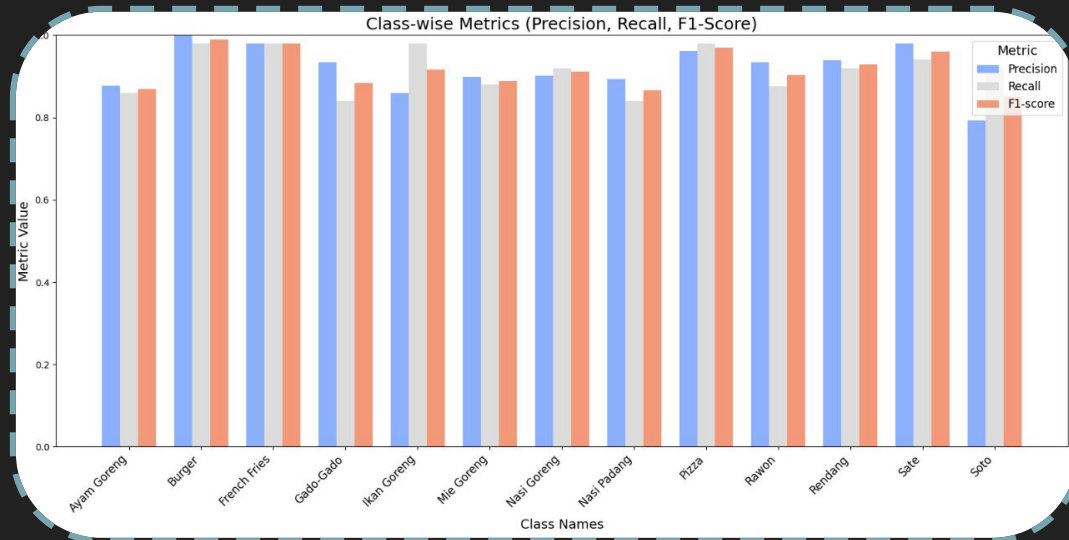


Layer 282: activation_89, Trainable: False
Layer 283: conv2d_86, Trainable: False
Layer 284: conv2d_90, Trainable: False
Layer 285: batch_normalization_86, Trainable: False
Layer 286: batch_normalization_90, Trainable: False
Layer 287: activation_86, Trainable: False
Layer 288: activation_90, Trainable: False
Layer 289: conv2d_87, Trainable: False
Layer 290: conv2d_88, Trainable: False
Layer 291: conv2d_91, Trainable: True
Layer 292: conv2d_92, Trainable: True
Layer 293: average_pooling2d_8, Trainable: True
Layer 294: conv2d_85, Trainable: True
Layer 295: batch_normalization_87, Trainable: True
Layer 296: batch_normalization_88, Trainable: True
Layer 297: batch_normalization_91, Trainable: True
Layer 298: batch_normalization_92, Trainable: True
Layer 299: conv2d_93, Trainable: True
Layer 300: batch_normalization_85, Trainable: True
Layer 301: activation_87, Trainable: True
Layer 302: activation_88, Trainable: True
Layer 303: activation_91, Trainable: True
Layer 304: activation_92, Trainable: True
Layer 305: batch_normalization_93, Trainable: True
Layer 306: activation_85, Trainable: True
Layer 307: mixed9_1, Trainable: True
Layer 308: concatenate_1, Trainable: True
Layer 309: activation_93, Trainable: True
Layer 310: mixed10, Trainable: True

Coding Approach - Step 4: Evaluate Model

- **Evaluation - Supervised Learning:**
 - *Precision: $TP / (TP + FP)$*
 - *Recall: $TP / (TP + FN)$*
 - *F1-Score: Mean of precision and recall*

Class-wise Metrics

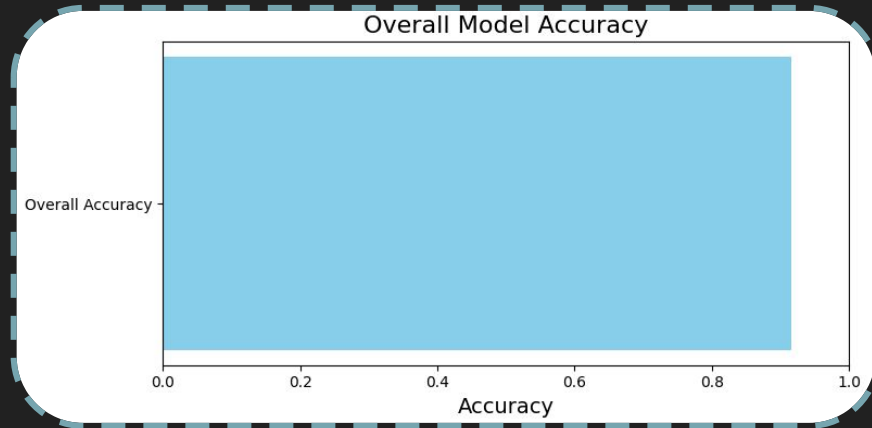


Overall Accuracy: 92%

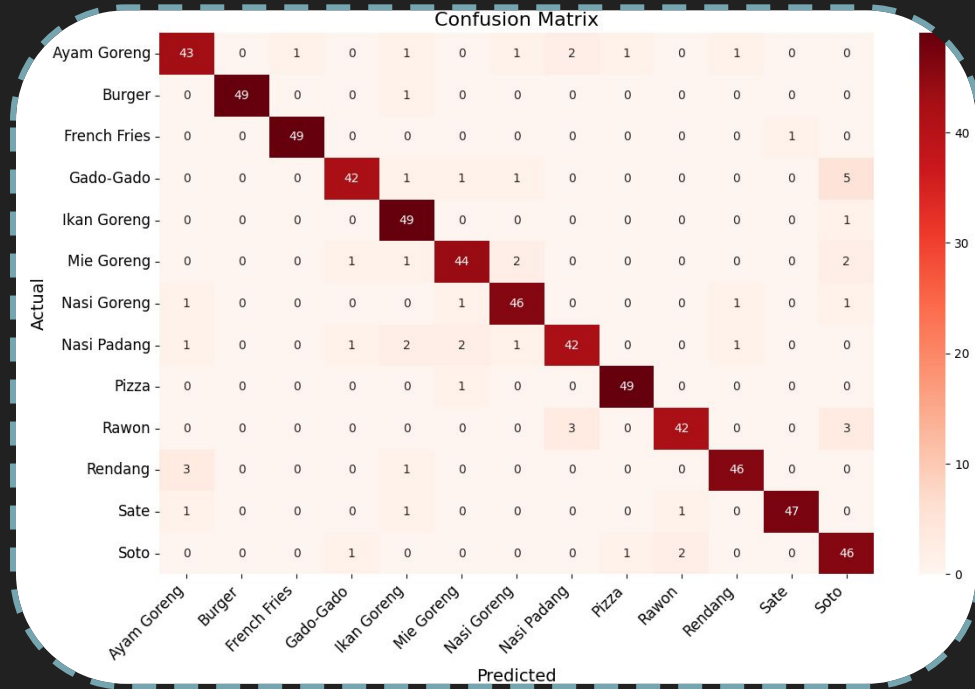
The highest overall accuracy achieved, when running all 6500 images and 20 epochs

Runtime Execution: 30 minutes

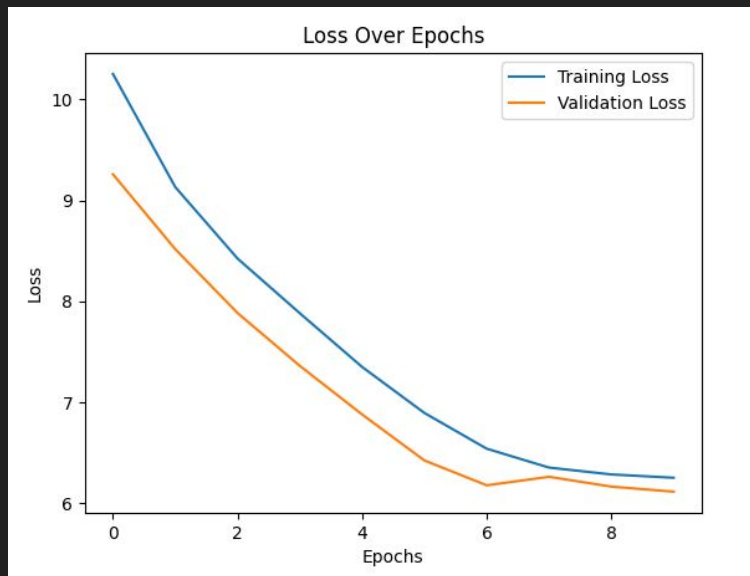
$$\text{Overall Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}}$$



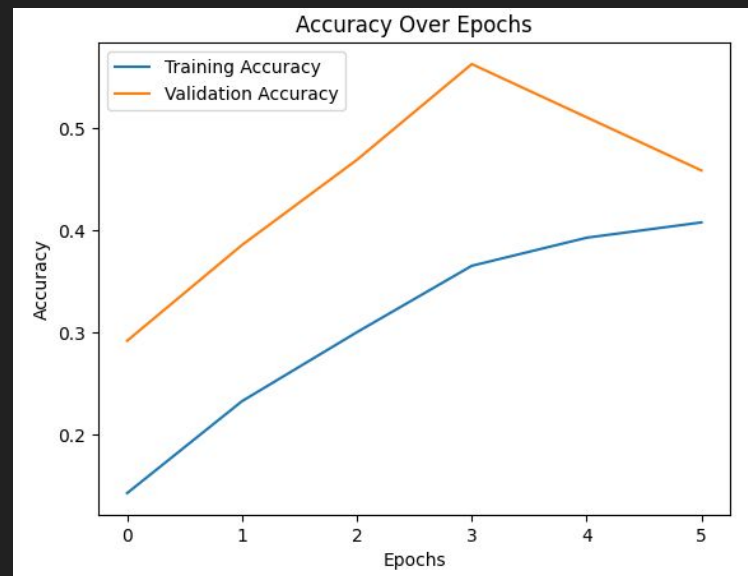
Confusion Matrix



Loss Over Epochs



Accuracy Over Epochs



*Thank you for your time
Are there any questions?*

